Assignment Overview

- **Objectives:** You are the lead data scientist of Taipei's YouBike operations centre. Every weekday at 8 am, stations near MRT exits run empty within minutes, while bikes pile up elsewhere. Rebalancing trucks must be dispatched *before* the rush, so the centre needs a model that predicts how many bikes each station will rent in each hour. Your job is to build an end-to-end **regression** pipeline, engineer features that capture commuting and weather, understand how **feature scaling** affects linear models, and use **learning curves** to diagnose high bias (underfitting) and high variance (overfitting).
- **Constraints:**
  1. Use **linear models only**: `LinearRegression`, `Ridge`, `Lasso`, and `SGDRegressor` from `scikit-learn`, optionally with `PolynomialFeatures`. Tree ensembles and neural networks are **not** allowed.
  2. Fit every scaler and encoder on training data only, inside a `sklearn.pipeline.Pipeline`.
  3. All code must run on Google Colab (free tier CPU is sufficient). Start from the provided guided notebook `A2_starter.ipynb`, do not clear its outputs before submitting, and fix all random seeds to `0`.
- **General Instructions:** The assignment consists of three compulsory components:
  1. **Part I:** Least Squares by Hand (20 Points)
  2. **Part II:** Feature Engineering and Scaling (45 Points)
  3. **Part III:** Learning Curves and Forecasting (35 Points)
- **Submission Guidelines:** Submit your work through the e3 submission system, including the following files:
  1. `A2_[ID].ipynb`: Your completed notebook with all outputs.
  2. `A2_[ID].pdf`: A PDF report containing your solutions to Part I (handwritten or typeset), all requested tables and plots, and your written analysis. Include a link to your Google Colab notebook (make sure it is viewable).

  You must also upload your test set predictions to the class leaderboard (Part III).
- **Late Submission Policy:**
  1. `25% penalty`: From 15 minute to 1 day late
  2. `50% penalty`: From 1 day to 2 days late
  3. `100% penalty`: More than 2 days late

Part I: Least Squares by Hand [20 marks]

Before dispatching a fleet of trucks, every operator practises on a single station. In this part you will fit, regularize, and rescale a tiny regression model by hand.

1. **Notation.** Let $X \in \mathbb{R}^{N \times d}$ be the design matrix and $\mathbf{y} \in \mathbb{R}^N$ the targets. Ordinary least squares (OLS) and ridge regression minimize, respectively,

$$J_{\text{OLS}}(\mathbf{w}) = \frac{1}{2}\|\mathbf{y} - X\mathbf{w}\|_2^2, \qquad J_{\text{ridge}}(\mathbf{w}) = \frac{1}{2}\|\mathbf{y} - X\mathbf{w}\|_2^2 + \frac{\lambda}{2}\|\mathbf{w}\|_2^2, \quad \lambda > 0.$$

   For a single feature x with intercept w_0 and slope w_1, let $S_{xx} = \sum_i (x^{(i)} - \bar{x})^2$ and $S_{xy} = \sum_i (x^{(i)} - \bar{x})(y^{(i)} - \bar{y})$. The intercept is *not* penalized, so the ridge slope is $w_1 = S_{xy}/(S_{xx} + \lambda)$.

2. **The Toy Dataset.** One station near an MRT exit was observed during five morning hours. The feature is precipitation x (mm) during the hour, and the target y is the number of bikes rented.

Hour	1	2	3	4	5
Precipitation x (mm)	0	1	2	3	4
Rentals y	10	8	7	4	1

3. **Tasks.**

   (a) **OLS and Ridge (10 points):**

   - Derive the ridge solution $\mathbf{w}^* = (X^\top X + \lambda I)^{-1} X^\top \mathbf{y}$ from $\nabla_{\mathbf{w}} J_{\text{ridge}} = \mathbf{0}$, and explain why it is unique for every $\lambda > 0$ even when $X^\top X$ is singular.
   - On the toy dataset, compute the OLS fit (w_0, w_1, MSE, and R^2) and the ridge slope for $\lambda = 1$ and $\lambda = 10$. Show your working, and describe what happens to the slope as $\lambda \to \infty$.

   (b) **Units Matter (10 points):** A new rain gauge reports precipitation in centimetres, so every $x^{(i)}$ is divided by 10.

   - Recompute the OLS slope and show that the OLS *predictions* do not change.
   - Recompute the ridge slope with $\lambda = 1$, convert it back to an effect per millimetre, and compare it with (a). Explain why ridge regression is *not* invariant to feature scaling, and why gradient descent on unscaled features converges slowly or diverges.

Part II: Feature Engineering and Scaling [45 marks]

The operations centre has shared one month of real rental logs. You will turn raw timestamps and weather readings into features that capture how Taipei commutes.

1. **The Dataset: Taipei YouBike 2.0 Hourly Demand (Jan-Jun 2026).** We built a station-by-hour dataset for the 30 busiest stations from four open sources:

   - **Rentals:** the Taipei City Department of Transportation's YouBike 2.0 trip records (Government Open Data License[1]). Each record is one trip (rental time, rental station, return time, return station, duration, bike type, date), with times published **rounded down to the hour**. We counted trips per rental station and hour, filling hours without trips with 0.
   - **Weather:** hourly observations at the Central Weather Administration's Taipei station (466920) from CODiS[2]. Trace rain (`T`, below 0.5 mm) is set to 0, missing readings are interpolated, and all stations share the same hourly weather.
   - **Calendar:** the government office calendar[3] (weekends and public holidays).
   - **Stations:** district and number of docks from the current YouBike 2.0 station list[4].

   The resulting files are on e3:

   - **Columns:** `id`, `datetime` (start of the hour), `station`, `district`, `capacity`, `is_holiday`, `temperature` (°C), `precipitation` (mm), `humidity` (%), `wind_speed` (km/h), and the target `rentals` (trips starting at the station in that hour).
   - **Files:** `train.csv` (1 to 24 March, with the target), `test.csv` (25 to 31 March, without the target), and `sample_submission.csv`.
   - **Validation:** the starter notebook holds out the last five days of `train.csv` (20 to 24 March) as the validation set. Report RMSE and R^2 on this set unless stated otherwise.

   No hour, weekday, or peak time columns are provided: deriving them from `datetime` is part of your job.

2. **Tasks.**

   (a) **Exploration and Baseline (10 points):**
   - Explore how demand varies over time, across stations, and with the weather, using plots of your choice. Summarise the most important patterns in a few sentences.
   - Report the training and validation RMSE and R^2 of the provided baseline (`LinearRegression` on the four weather columns). Decide, with justification, whether to model a transformed target (e.g., $\log(1+y)$) instead of y.

   (b) **Feature Engineering (20 points):** Starting from the baseline, design and evaluate a sequence of at least three **cumulative** feature sets, each motivated by a pattern you found in (a). Your sets should go beyond the raw columns, for example by deriving time features from `datetime`, encoding categorical variables, or adding interactions; the choice of features and encodings is yours.
   - Keep the model fixed across sets (e.g., `Ridge` with one regularization strength of your choice), and report the number of features, training RMSE, validation RMSE, and validation R^2 for each set in one table.
   - Explain which features helped most and why. For every encoding choice you made (e.g., how you represented time), justify it from the perspective of a *linear* model.

   (c) **Feature Scaling (15 points):** Choose one of your feature sets and compare at least three preprocessing options for the numeric columns (including no scaling). Train at least one model with a closed form solution, one regularized model, and one model fitted by gradient descent (e.g., `SGDRegressor`), with hyperparameters of your choice that make the effect of scaling visible.
   - Report the validation RMSE of every model and scaling combination in one table, together with the number of gradient descent iterations.
   - Explain each difference you observe using Part I(b).

[1] https://data.taipei/dataset/detail?id=c5924c17-25db-4f1e-99c4-f8ada40f2445
[2] https://codis.cwa.gov.tw/
[3] https://data.gov.tw/dataset/14718
[4] https://tcgbusfs.blob.core.windows.net/dotapp/youbike/v2/youbike_immediate.json

Part III: Learning Curves and Forecasting [35 marks]

A model that looks good on one split may still be starved of data or drowning in parameters. In this part, you will diagnose your models, fix them, and face the final week of March.

1. **Setup.** Use the provided `plot_learning_curve` helper, which plots training and validation RMSE against the number of training examples (with cross validation) for any `scikit-learn` pipeline. Choose two models of your own:
   - **Model A:** a deliberately **simple** model that you expect to underfit;
   - **Model B:** a deliberately **flexible** model that you expect to overfit, e.g., using polynomial or interaction features without regularization.
2. **Tasks.**
   (a) **Diagnosis (15 points):** Describe both models and plot their learning curves side by side. For each model, state whether it suffers from high bias or high variance, citing the final level of both curves and the gap between them, and state whether collecting more data would help.
   (b) **Remedies (10 points):** Apply **one** remedy to each model that matches your diagnosis (e.g., richer features for high bias, or regularization with a tuned strength or fewer features for high variance). Redraw the learning curves and explain whether each remedy worked.
   (c) **Leaderboard (10 points):** Select your final model using the validation set **only**, retrain it on all of `train.csv`, predict `test.csv` (clipping negative predictions to zero), and upload `submission.csv` to the class leaderboard. Report your test RMSE, which must beat the TA baseline (`LinearRegression` on the weather columns) for full marks, and comment in two or three sentences on whether your validation RMSE was a reliable estimate.